

Answers

1. True/False: The lerp function, combined with a shaping function, provides a flexible way to customize animations.

Answer: True

2. True/False: When using lerp on a ball's position, by compressing the graph near 0, the ball starts slow but then accelerates.

Answer: True

3. True/False: The smoothstep function applies the equation $3t^2 - 2t^3$

Answer: True

4. True/False: Raising “ t ” to a power greater than 1 results in a fast start for the animation.

Answer: False

5. True/False: The term “easeIn” refers to animations that slow down at the start.

Answer: True

6. True/False: The functions t^2 and t^3 could be considered “easeOut” functions

Answer: False

7. True/False: Raising “ t ” to a power less than 1 results in an “easeOut” style function.

Answer: True

8. True/False: It's valid to lerp between shaping functions to create a new shaping function. ie. `lerp(t * t, smoothstep(0.0, 1.0, t), t)`

Answer: True, there's really nothing stopping you from doing this, in fact it's a powerful way to compose functions together.

9. Implement `smoothstep(minValue, maxValue, value)` using `inverseLerp`, where `minValue` specifies the start of the range, and `maxValue` specifies the end of the range.

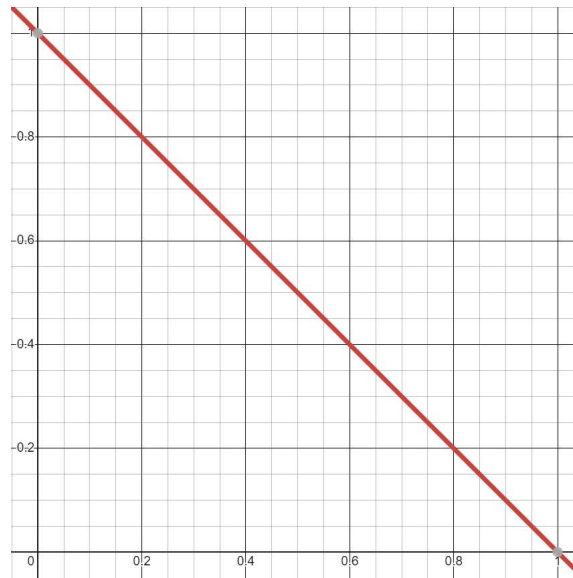
Answer: `float t = inverseLerp(minValue, maxValue, value);`
 `t = clamp(t, 0, 1);`
 `return 2 * t * t - 3 * t * t * t;`

The general idea should be to use `inverseLerp` to generate the `t` value, clamp it to the range `[0, 1]`, and then apply the `smoothstep` function. If

you happen to skip the clamp or assume that inverseLerp implements it, that's fine, as long as you understand the general idea here.

10. What does `inverseLerp(1.0, 0.0, t)` do?

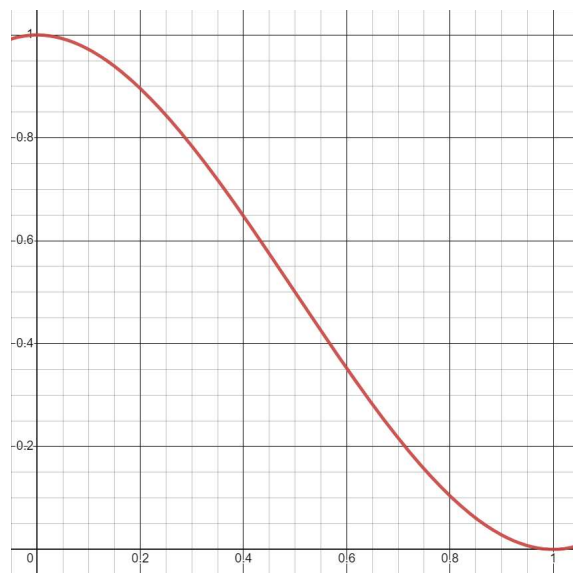
Answer: Basically reverses the $y=x$ graph, so that it starts at 1 and heads to 0.



11. What does `smoothstep(1.0, 0.0, t)` do?

Answer: This basically reverses the smoothstep, so that it starts at 1 for $t = 0$, and smoothly transitions to 0 for $t = 1$.

It will end up looking like this:



12. Use smoothstep to implement a “pulse”, the pulse should start at $t = 0.25$, go up to 1 at $t = 0.5$, then back to 0 at $t = 0.75$. Use an “if” statement to construct your shaping function.

Answer: *It could look something like this:*

```
float pulse(float t) {  
    if (t < 0.5) {  
        return smoothstep(0.25, 0.5, t);  
    } else {  
        return smoothstep(0.75, 0.5, t);  
    }  
}
```

13. Use smoothstep to implement a “pulse” again, the pulse should start at $t = 0.25$, go up to 1 at $t = 0.5$, then back to 0 at $t = 0.75$. Do NOT use an “if” statement this time.

Answer: *Here’s an easy way:*

$\text{smoothstep}(0.25, 0.5, t) * \text{smoothstep}(0.75, 0.5, t);$

14. What happens if you have 2 shaping functions. The first is t^2 , while the 2nd is $1 - (1 - t)^2$, and you lerp between those using t ?

ie. $\text{lerp}(t^2, 1 - (1 - t)^2, t)$?

Fully expand this out and explain the result.

Answer: *Plugging this into the lerp equation, which is $a + t * (b - a)$*

You have

$\text{lerp}(t^2, 1 - (1 - t)^2, t)$

$$a = t^2$$

$$b = 1 - (1 - t)^2$$

$$b = 1 - (1 - 2t + t^2)$$

$$b = 2t - t^2$$

$$= t^2 + t * (2t - t^2 - t^2)$$

$$= t^2 + 2t^2 - 2t^3$$

$$= 3t^2 - 2t^3$$

We just built smoothstep from 2 functions!